

Estimating Mean Score Reliability with RMU

Giacomo Bignardi

2025-10-02

Introduction

i Note

This tutorial can be run and tested inside the docker image `bignardig/tidyverse461:v5`, which has all the required R packages pre-installed.

This tutorial demonstrates how to use Relative Measurement Uncertainty (RMU) to estimate mean score reliability. See our pre-print for more information on RMU.

This brief tutorial assumes your relatively familiar with multilevel Bayesian modelling and logistic regression.

Set Up

Imagine we measure the length of 5000 mice 5 times each. Every time we measure a mouse, we will get a slightly different measurement (have you tried holding a mouse with one hand and holding a ruler in another?).

In this example, imagine that each measurement from each subject follows a normal distribution with a fixed mean (called the “true score” in psychometrics; θ_i) and fixed variance (σ_ϵ^2). Assume that this measurement error variance (σ_ϵ^2) is the same across all participants.

The final important parameter to consider is the variance in mouse true scores across the mouse population (σ_θ^2).

To illustrate, let’s simulate some data:

```
library(ggdist)
library(brms)
library(cmdstanr)
library(psych)
library(tidyverse)
library(gbtoolbox)

rm(list=ls())
```

```

set.seed(1)
N          = 5000 # number of subjects (mice)
J          = 5    # number of measurements per subject
true_score_variance = 3
error_variance      = 2

df = expand.grid(j = 1:J, mouse = 1:N)

true_scores      = rnorm(N, mean = 10, sd = sqrt(true_score_variance))
measurement_error = rnorm(N*J, mean = 0, sd = sqrt(error_variance))

df$measurement = true_scores[df$mouse] + measurement_error

df_average_lengths = df |>
  group_by(mouse) |>
  summarise(average_measurement = mean(measurement))

```

In the above example, we can work out the reliability of the mean score using J , σ_ϵ^2 and σ_θ^2 :

$$\begin{aligned}
 R(\hat{y}) &= \frac{\sigma_\theta^2}{\sigma_\theta^2 + \sigma_\epsilon^2 / J} \\
 &= \frac{3}{3 + 2/5} = 0.882
 \end{aligned}$$

In case you don't trust me, the derivation for this formula is in the appendix. However, we can also check its correct by computing the squared correlation between the observed mean scores and true scores:

```
cor(df_average_lengths$average_measurement, true_scores)^2
```

```
[1] 0.8902067
```

Estimating Reliability

It's worth noting that the Bayesian measurement model we employ in this example effectively makes the same assumptions as coefficient alpha - which if violated can lead to bias (this paper covers the assumptions of alpha nicely).

However, with RMU, we can make our model more complex (e.g., use an alternative outcome distribution, add parameters, etc.) to account for any quirks with our measurements. For example, for modelling reaction times, you may want to use a more appropriate distribution.

We will keep our model simple in this example - but if you want to apply this code to real data - its worth considering whether the model assumptions are met!

(1) Coefficient alpha

We can estimate reliability using (unstandardised) coefficient alpha. First, we need to pivot_wider our dataset so that each row represents a mouse and our five measurements are on five columns:

```
df_wide = df |>
  pivot_wider(
    id_cols = mouse,
    names_from = j,
    values_from = measurement,
    names_prefix = "measurement"
  )

df_wide |>
  head()
```

```
# A tibble: 6 × 6
  mouse measurement1 measurement2 measurement3 measurement4 measurement5
  <int>          <dbl>          <dbl>          <dbl>          <dbl>          <dbl>
1     1           6.77           9.80           6.54           10.6           10.5
2     2           8.57           8.58          11.2           10.7           10.2
3     3           7.41           8.71           8.30           8.63           10.4
4     4          12.1           11.6          12.0           12.4           11.9
5     5          11.1          12.6          12.5           11.2           10.2
6     6           6.54           7.19           6.89           8.49           11.3
```

Coefficient alpha calc also gives us .88!

```
alpha = psych::alpha(df_wide[-1])$total$raw_alpha

alpha_ci = psych::alpha.ci(alpha = alpha, n.obs = N, n.var = J, digits = 4)

alpha_ci
```

```
95% confidence boundaries (Feldt)
lower alpha upper
0.88 0.89 0.89
```

(2) RMU

To calculate RMU, we need to first fit a Bayesian measurement model, and then extract a matrix of posterior draws for each mouse's length parameter.

We can use a simple random intercept multilevel model, as shown below.

To fit the model, we use a “long” dataset which has a column for measurement, and a column to indicate mouse. Note that multiple measurements from each mouse are stored in the same column (measurement).

```
head(df)
```

```
  j mouse measurement
1 1     1    6.770474
2 2     1    9.804690
3 3     1    6.541625
4 4     1   10.583413
5 5     1   10.495552
6 1     2    8.567657
```

Fit brms Multilevel Model

Let's fit our model:

```
set.seed(1)

brms_model = brm(
  measurement ~ 1 + (1 | mouse),
  data      = df,
  chains    = 4,
  cores     = 4,
  backend   = "cmdstanr", # This is to speed up model fitting using parallelization
  threads   = threading(2) # This is to speed up model fitting using parallelization
)
```

And look at our results:

```
summary(brms_model)
```

```
Family: gaussian
Links: mu = identity
Formula: measurement ~ 1 + (1 | mouse)
Data: df (Number of observations: 25000)
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
       total post-warmup draws = 4000

Multilevel Hyperparameters:
~mouse (Number of levels: 5000)
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
```

```
sd(Intercept)      1.76      0.02      1.73      1.80 1.01      887      1906
```

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	9.99	0.03	9.94	10.04	1.00	979	1892

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	1.42	0.01	1.41	1.43	1.00	3568	2782

Draws were sampled using `sample(hmc)`. For each parameter, `Bulk_ESS` and `Tail_ESS` are effective sample size measures, and `Rhat` is the potential scale reduction factor on split chains (at convergence, `Rhat = 1`).

Our fitted model has the following structure (rounded to whole numbers):

$$\begin{aligned}
 measurement_{ij} &= 10 + \theta_j + \epsilon_{ij} \\
 \theta_j &\sim N(0, 3) \quad (\text{mouse random-intercepts}) \\
 \epsilon_{ij} &\sim N(0, 2) \quad (\text{measurement error})
 \end{aligned}$$

Calculate RMU

To calculate RMU, extract the posterior draws for each mouse's random intercept parameter, and plug the matrix of draws into my reliability function.

`library(gbtoolbox)` is already loaded above (see Set Up), but if you don't have it installed, you can either install my R package `gbtoolbox` from github, or you can just load the reliability function directly from github (not run here, since `gbtoolbox` is already installed in the docker image used to build this tutorial). Both methods are outlined below:

Install R package from github:

```
devtools::install_github("giac01/gbtoolbox")
```

Or, you can just load the R function using the following code (but make sure you also have the R package `ggdist` installed):

```
source("https://raw.githubusercontent.com/giac01/gbtoolbox/refs/heads/main/R/
reliability.R")
```

```
posterior_draws = brms_model |>
  as_draws_df() |>
  select(starts_with("r_mouse")) |>
  t()
```

```
nrow(posterior_draws) # should equal the number of subjects
```

```
[1] 5000
```

```
ncol(posterior_draws) # should equal the number of post-warmup posterior draws
```

```
[1] 4000
```

```
gbtoolbox::reliability(posterior_draws)
```

```
Reliability (Relative Measurement Uncertainty)
```

```
Number of subjects:          5000
```

```
Number of input posterior draws: 4000
```

```
Number of RMU posterior draws:  2000
```

```
RMU estimate: 0.885
```

```
Posterior SD: 0.004
```

```
95% HDCI:      [0.881, 0.890]
```

```
Access full results with $hdci and $reliability_posterior_draws
```

Remember to check that the matrix of draws has subjects on the rows and draws on the columns. We can check this because we know we have 5000 subjects, and the function has confirmed this is the case.

Recap of simulation code

Just to check this wasn't a fluke, we can play around with the parameters `J` and `true_score_variance` and `error_variance` below to show that we get the correct result across different simulation conditions:

```
rm(list=ls())
set.seed(1)
N = 5000 # number of subjects (mice)
J = 3    # number of measurements per subject
true_score_variance = 1
error_variance = 10

df = expand.grid(j = 1:J, mouse = 1:N)
```

```

true_scores      = rnorm(N, mean = 10, sd = sqrt(true_score_variance))
measurement_error = rnorm(N*J, mean = 0, sd = sqrt(error_variance))

df$measurement = true_scores[df$mouse] + measurement_error

df_average_lengths = df |>
  group_by(mouse) |>
  summarise(average_measurement = mean(measurement))

cat("Simulation reliability = ")

true_score_variance/(true_score_variance+error_variance/J)

cat("Squared Correlation between mean and true scores = ")
# This may slightly differ to above due to sampling variance

cor(df_average_lengths$average_measurement, true_scores)^2

# Fit model and calculate RMU
# I've reduced the number of MCMC chains here to two for speed - you may want
more in practice!

brms_model = brm(
  measurement ~ 1 + (1 | mouse),
  data      = df,
  chains    = 2,
  cores     = 2,
  refresh   = 0,          # This hides the progress bar
  backend   = "cmdstanr", # This is to speed up model fitting using parallelization
  threads   = threading(4) # This is to speed up model fitting using parallelization
)

posterior_draws = brms_model |>
  as_draws_df() |>
  select(starts_with("r_mouse")) |>
  t()

gbtoolbox::reliability(posterior_draws)

```

For the above example, RMU is pretty similar to the expected value:

```

cat("Simulation reliability = ")

```

```
Simulation reliability =
```

```
true_score_variance/(true_score_variance+error_variance/J)
```

```
[1] 0.2307692
```

```
cat("Squared Correlation between mean and true scores = ")
```

```
Squared Correlation between mean and true scores =
```

```
# This may slightly differ to above due to sampling variance
```

```
cor(df_average_lengths$average_measurement, true_scores)^2
```

```
[1] 0.2414736
```

```
cat("RMU:\n")
```

```
RMU:
```

```
gbtoolbox::reliability(posterior_draws)
```

```
Reliability (Relative Measurement Uncertainty)
```

```
Number of subjects:          5000
```

```
Number of input posterior draws: 2000
```

```
Number of RMU posterior draws: 1000
```

```
RMU estimate: 0.251
```

```
Posterior SD: 0.018
```

```
95% HDCI:      [0.217, 0.282]
```

```
Access full results with $hdci and $reliability_posterior_draws
```

Beyond coefficient alpha

We can leverage the flexibility of brms + RMU to estimate reliability in more complex scenarios.

In this next example, we'll simulate binary outcome data (1 = success, 0 = fail) where:

- Each subject completes between 3-9 trials (varying number of measurements per subject)
- Subjects improve with practice (probability of success increases on each subsequent trial)
- We're interested in individual differences in first-trial performance

Because subjects completed different numbers of trials, simply averaging their scores would be misleading - those who completed more trials had more opportunities to practice and improve.

Instead, we'll use a multilevel logistic regression model to estimate each subject's probability of success on the first trial. We'll include trial number as fixed covariate in the model. We'll code trial number (j) starting at 0 for the first trial, so the random intercept directly represents each subject's first-trial performance (on the logit scale).

```
rm(list=ls())
set.seed(1)
N = 5000 # number of subjects (mice)
J = sample(3:9, size = N, replace = TRUE) # number of
measurements per subject
true_score_variance = 1
error_variance = 2

df = expand.grid(j = 1:10, mouse = 1:N)

for(i in 1:N){
  df = df |>
    filter(!(mouse == i & j > J[i]))
}

prob_success_first_trial = rbeta(N,3,3) # probability of success on first trial

df$prob_success_first_trial = prob_success_first_trial[df$mouse] |> qlogis()

df$y = rbinom(nrow(df), size = 1, prob = plogis(df$prob_success_first_trial
+ .1*df$j))

df$j = df$j - 1

brms_model_logistic = brm(
  y ~ 1 + j + (1 | mouse),
  data = df,
  family = bernoulli(link = "logit"),
  chains = 4,
  cores = 4,
  backend = "cmdstanr",
```

```

    threads = threading(2)
)

```

Our data structure looks like this:

```

df |>
  select(-prob_success_first_trial) |>
  slice(1:10)

```

```

      j mouse y
1  0      1 1
2  1      1 0
3  2      1 1
4  0      2 1
5  1      2 1
6  2      2 1
7  3      2 1
8  4      2 0
9  5      2 1
10 0      3 1

```

Fitted brms model summary:

```
summary(brms_model_logistic)
```

```

Family: bernoulli
Links: mu = logit
Formula: y ~ 1 + j + (1 | mouse)
Data: df (Number of observations: 30080)
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
       total post-warmup draws = 4000

Multilevel Hyperparameters:
~mouse (Number of levels: 5000)

      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sd(Intercept)    0.88     0.02    0.84    0.92 1.00    1392    2034

Regression Coefficients:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept    0.10     0.02    0.05    0.14 1.00     5664     3284
j             0.10     0.01    0.09    0.11 1.00     8061     2734

```

Draws were sampled using `sample(hmc)`. For each parameter, `Bulk_ESS` and `Tail_ESS` are effective sample size measures, and `Rhat` is the potential scale reduction factor on split chains (at convergence, `Rhat = 1`).

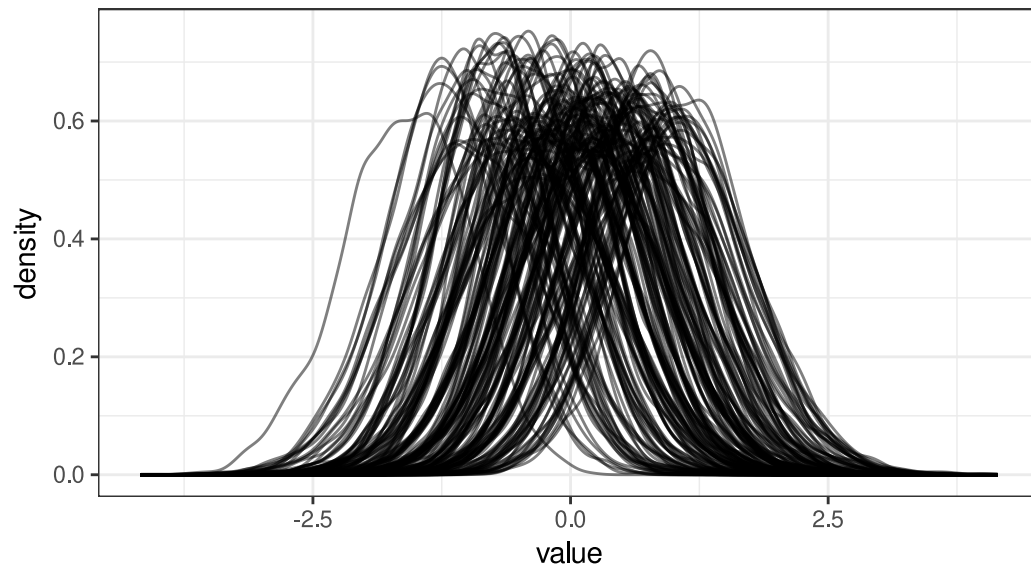
We visualise the posterior distribution for each subject's random intercept below.

By transforming the random intercept with the inverse logit function we can look at our posterior on the probability scale too - these values tell us the estimated probability of success on the first trial for each subject.

```
posterior_draws = brms_model_logistic |>
  as_draws_df() |>
  select(starts_with("r_mouse")) |>
  t()

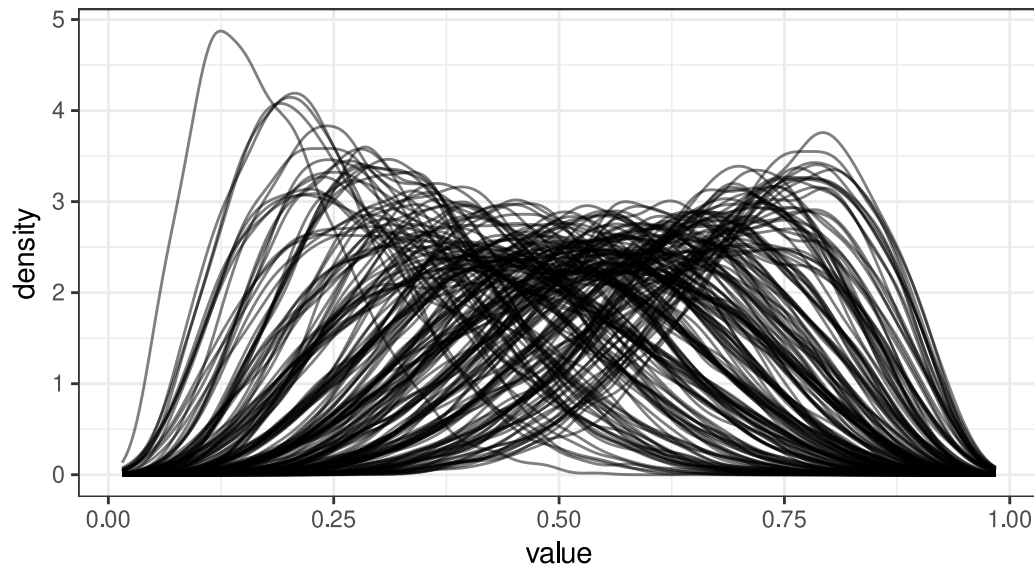
posterior_draws |>
  t() |> # make sure participant is on the COLUMN for pivot_wider!
  data.frame() |>
  (\(x) `colnames`<-`(x, 1:ncol(x))`()) |>
  pivot_longer(cols = everything()) |>
  filter(as.numeric(name) < 201) |>
  ggplot(aes(x = value, group = name)) +
  geom_density(fill = NA, color = ggplot2::alpha("black", .5)) +
  theme_bw() +
  guides(color = "none") +
  labs(
    subtitle = "Random Intercept Posterior Distributions for each subject \nLogit
scale, first 200 subjects only"
  )
```

Random Intercept Posterior Distributions for each subject
Logit scale, first 200 subjects only



```
posterior_draws |>
  t() |> # make sure participant is on the COLUMN for pivot_wider!
  data.frame() |>
  (\(x) `colnames<-`(x, 1:ncol(x)))() |>
  pivot_longer(cols = everything()) |>
  filter(as.numeric(name) < 201) |>
  mutate(value = plogis(value)) |>
  ggplot(aes(x = value, group = name)) +
  geom_density(fill = NA, color = ggplot2::alpha("black", .5)) +
  theme_bw() +
  labs(
    subtitle = "Random Intercept Posterior Distributions for each subject\nProbability scale, first 200 subjects only"
  )
```

Random Intercept Posterior Distributions for each subject
Probability scale, first 200 subjects only



To find the “true” reliability, we can calculate the squared correlation between the estimated probabilities of success for each subject (using posterior means for the random intercepts) and the known probabilities of success.

Rather than working directly with the probabilities, we will calculate the correlation on the logit scale.

```
posterior_means = rowMeans(posterior_draws)

cor(posterior_means,prob_success_first_trial)^2
```

```
[1] 0.4724863
```

```
gbtoolbox::reliability(posterior_draws)
```

Reliability (Relative Measurement Uncertainty)

Number of subjects: 5000

Number of input posterior draws: 4000

Number of RMU posterior draws: 2000

RMU estimate: 0.475

Posterior SD: 0.012

95% HDI: [0.454, 0.497]

Access full results with `$hdci` and `$reliability_posterior_draws`

Our RMU estimate of reliability closely matches the squared correlation between estimated and true scores - which is reassuring!

A more robust approach to find the “true” reliability would be to simulate data from two time points for each participant - and show that the correlation between estimates from each time point is equivalent to RMU. In the case that each subject’s *expected* score corresponds to `prob_success_first_trial`, then either approach will yield the same result, however this is not always the case.

Appendix: Deriving The Mean Score Reliability Formula

Consider a measurement model where we measure N participants, and we measure each participant J times (or across J items). Each participant has a true score θ_i which represents the *expected* measurement for participant i .

We assume that measurements from a given subject follow a normal distribution with mean (θ_i) and variance (σ_ϵ^2).

We assume that the distribution of true scores across the population is normally distributed with mean μ and variance σ_θ^2 .

We can describe the distribution of each measurement as follows:

$$\begin{aligned}
 \text{measurement } j \text{ for subject } i &= \text{subject } i \text{ true score} + \text{measurement error} \\
 \underbrace{y_{ij}} &= \underbrace{\theta_i} + \underbrace{\epsilon_{ij}} \\
 &\quad \text{error variance} \\
 \epsilon_{ij} &\sim \text{Normal} (0, \underbrace{\sigma_\epsilon^2}) \\
 \underbrace{\theta_1, \theta_2, \dots, \theta_N}_{\text{population true scores}} &\sim \text{Normal} (\underbrace{\mu}_{\text{population mean true score}}, \underbrace{\sigma_\theta^2}_{\text{true score variance}})
 \end{aligned}$$

Calculating Reliability of Mean Scores

Given that we know the distribution of each measurement, the distribution of a mean of multiple independent measurements is also Normal but with a smaller variance (note that as J increases, the variance decreases):

$$\begin{aligned}
\hat{y}_i &= \frac{1}{J} \sum_{j=1}^J y_{ij} \\
&= \frac{1}{J} \sum_{j=1}^J (\theta_i + \epsilon_{ij}) \\
&= \theta_i + \sum_{j=1}^J \frac{\epsilon_{ij}}{J} \\
&= \theta_i + \mathcal{E}_i \\
\mathcal{E}_i &= \frac{\epsilon_{i1} + \epsilon_{i2} + \dots + \epsilon_{iJ}}{J} \sim \text{Normal} \left(0, \frac{\sigma_\epsilon^2}{J} \right)
\end{aligned}$$

To work out the variance of the mean score for subject i , note that the variance of the sum of J independent normally distributed variables $(\epsilon_{i1} + \epsilon_{i2} + \dots + \epsilon_{iJ})$ is equal to $J \sigma_\epsilon^2$, and dividing by a constant J divides the variance by J^2 .

Reliability Formula

Reliability is the ratio of true score variance divided by the observed score variance:

$$\begin{aligned}
R(\hat{\mathbf{y}}) &= R(\boldsymbol{\theta} + \mathcal{E}) \\
&= \frac{\text{Var}(\boldsymbol{\theta})}{\text{Var}(\boldsymbol{\theta}) + \text{Var}(\mathcal{E})} \\
&= \frac{\sigma_\theta^2}{\sigma_\theta^2 + \sigma_\epsilon^2 / J}
\end{aligned}$$

This formula shows that reliability increases as we add more items (J increases), because the error variance gets divided by J .